Résumé : Programmation parallèle avec OpenMP

1 Définition

OpenMP (*Open MultiProcessing*) est une API de programmation à base de **directives** pour écrire des programmes parallèles sur des systèmes à **mémoire partagée**.

— L'approche la plus simple consiste à **paralléliser les boucles** : plusieurs threads sont lancés, chacun exécutant une ou plusieurs itérations.

— Interface disponible pour les langages **C, C++ et Fortran**, supportée par de nombreux environnements (Linux, Windows, ...).

2 Modèle d'exécution : fork-join

— L'exécution commence avec un seul thread : le **thread maître**.

— Début d'une construction parallèle (*fork*) : le thread maître crée un ensemble de threads.

— Fin de la construction parallèle (*join*) : les threads se synchronisent via une **barrière implicite**.

— Seul le thread maître continue ensuite son exécution (région séquentielle).

Notion de thread. Un thread est un « processus léger » : un processus dans un processus. Les threads d'un même processus **partagent la mémoire virtuelle**, alors que chaque processus possède la sienne.

3 Mise en œuvre

1. Inclure l'en-tête : `#include <omp.h>`
2. Compiler avec l'option `-fopenmp` : `gcc -fopenmp -o Mon_prog mon_source.c`
3. Fixer le nombre de threads : `export OMP_NUM_THREADS=2`

Mesure du temps d'exécution.

```
double temps_initial = omp_get_wtime();
/* Travail */
printf("Temps pris : %f\n", omp_get_wtime() - temps_initial);
```

Nombre de CPU. Par défaut, nombre de threads = nombre de CPU.

```
$ cat /proc/cpuinfo | grep processor | wc -l
```

```
#include <sched.h>
int nb = sched_getcpu(); // num\'ero du CPU courant
int nb = sysconf(_SC_NPROCESSORS_ONLN); // nombre de CPU en ligne
```

4 Directive de base et fonctions utiles

La directive `##pragma omp parallel` indique le début d'une **région parallèle**.

Fonction	Rôle
`omp_get_num_threads()`	nombre de threads en cours (dans une région parallèle)
`omp_get_thread_num()`	rang du thread courant
`omp_set_num_threads(nb)`	fixe le nombre de threads (si `num_threads()` n'est pas utilisé)
`omp_in_parallel()`	retourne 1 si on est dans une région parallèle, 0 sinon

Exemple 1 : Hello World parallèle

```c
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[]) {
  #pragma omp parallel
  {
    int rang = omp_get_thread_num();
    printf("Hello World du thread : %d !\n", rang);
    if (rang == 0)
      printf("Nombre total de threads : %d\n", omp_get_num_threads());
  }
  return 0;
}
```

5 Statut des variables

Clause	Description
`shared`	par défaut, toutes les variables sont partagées (sauf les compteurs d'itérations)
`private`	variable privée à chaque thread, **non initialisée**
`firstprivate`	privée, **initialisée** avec la valeur de la variable originale
`lastprivate`	privée ; la variable originale est mise à jour après modification
`threadprivate(i)`	privée, valeur qui **persiste** d'une région parallèle à une autre

6 Clauses de la directive `parallel`

```c
#pragma omp parallel num_threads(nb_th) // nombre de threads de la r\'egion
#pragma omp parallel if(expression) // threads cr\'e\'es seulement si expression != 0,
                                    // sinon ex\'ecution s\'equentielle
#pragma omp parallel default(shared|none) // none : oblige \`a d\'eclarer chaque
    variable
#pragma omp parallel copyin(i) // initialise une variable threadprivate
                               // avec la valeur du thread ma\^itre
```

— `copyprivate(list_var)` : copie les valeurs des variables privées d'un thread, modifiées dans une région `single`, vers les variables correspondantes des autres threads. Ne peut pas être utilisée avec `nowait`.

— `nowait` : annule les barrières implicites.

7 Synchronisation et exécution exclusive

Directive	Effet
`##pragma omp master`	région exécutée uniquement par le thread maître (rang 0) ; les autres threads continuent
`##pragma omp single`	région exécutée par un seul thread (le premier arrivé, imprévisible) ; les autres sont bloqués
`##pragma omp barrier`	synchronisation de tous les threads
`##pragma omp critical`	section exécutée par un seul thread à la fois
`##pragma omp atomic`	mise à jour atomique d'une variable (`++`, `--`, opération logique)

8 Parallélisation des boucles `for`

La variable de boucle `i` est privée par défaut. Deux écritures équivalentes :

```
#pragma omp parallel for
for (i = 0; i < N; i++) { ... }

#pragma omp parallel
{
  #pragma omp for
  for (i = 0; i < N; i++) { ... }
}
```

8.1 Clause `reduction`

Une copie locale de `res` est associée à chaque thread, initialisée selon l'opération `op` (0 pour l'addition). Les copies sont combinées à la fin.

```
#pragma omp parallel for reduction(op : res)
for (i = 0; i < N; i++) { ... }
```

8.2 Clause `ordered`

Le code de la directive `##pragma omp ordered` est exécuté dans l'**ordre séquentiel**. Elle doit être utilisée uniquement dans un `parallel for ordered` ou un `for ordered`.

```
#pragma omp parallel for ordered
for (i = 0; i < N; i++) {
  ...
  #pragma omp ordered
  printf("i=%d \n", i);
}
```

8.3 Clause `schedule`

Elle définit la **distribution des itérations** entre les threads :

```
#pragma omp parallel for schedule(type, taille_bloc)
```

avec `type` $\in$ {`static`, `dynamic`, `guided`, `runtime`, `auto`} et `taille_bloc` le nombre d'itérations par bloc.

Type	Comportement
`static`	Répartition par défaut : N/p itérations par thread (p = nombre de threads). Avec une taille de bloc, les blocs sont distribués de façon circulaire (*round-robin*), ex. `static, 3` : th_0 → itérations 1–3, th_1 → 4–6, ...
`dynamic`	Dès qu'un thread termine son bloc, il prend le suivant dans la liste d'attente. Adapté quand les itérations n'ont pas le même coût.
`guided`	Similaire à `dynamic`, mais la taille du bloc suivant dépend du nombre d'itérations restantes : $x = \frac{N}{p}$, $y = \frac{N-x}{p}$, $z = \frac{N-(x+y)}{p}$, ...
`runtime`	Le choix est précisé lors de l'exécution.
`auto`	Le choix est laissé au compilateur.

9 Sections

Partage de travail : chaque `section` est exécutée **en parallèle par un thread différent**.

```
#pragma omp sections
{
  #pragma omp section
  { ... }
  #pragma omp section
  { ... }
}
```

10 Exemple 2 : initialisation d'une matrice

Programme séquentiel : allocation d'une matrice $n \times n$ ($n = 9000$) et remplissage `a[i][j] = i*j`. Version parallèle pour deux threads :

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main() {
  int n = 9000, i, j;
  int **a = (int **) malloc(n * sizeof(int *));
  double t0 = omp_get_wtime();

  #pragma omp parallel for num_threads(2) private(j)
  for (i = 0; i < n; i++) {
    a[i] = (int *) malloc(n * sizeof(int));
    for (j = 0; j < n; j++)
      a[i][j] = i * j;
  }
  printf("Temps pris : %f\n", omp_get_wtime() - t0);

  for (i = 0; i < n; i++) free(a[i]);
  free(a);
  return 0;
}
```

Analyse : on compare les temps d'exécution des versions séquentielle et parallèle (avec `omp_get_wtime()`) ; avec 2 threads, on s'attend à un gain proche de 2 sur la partie parallélisée, limité par le coût de création des threads et par les accès mémoire (`malloc`).